

Master DSA Interview Cheat Sheet

A pattern-recognition cheat sheet: **When you see this → think of this technique.**

1. Arrays

Problem Pattern	Technique
Find pair with target	Two Sum / HashMap
Sorted array pair	Two Pointers
Remove duplicates	Two Pointers
Move zeros	Two Pointers
Maximum subarray sum	Kadane's Algorithm
Subarray with given sum	Sliding Window / Prefix Sum
Count subarrays with sum K	Prefix Sum + HashMap
Range sum queries	Prefix Sum
Product except self	Prefix/Suffix
Majority element	Boyer-Moore
Missing number	XOR / Math
Merge intervals	Sorting
Rotate array	Reverse
Next permutation	Pivot + Reverse

2. Strings

Problem Pattern	Technique
Check palindrome	Two Pointers
Reverse string	Two Pointers
Anagram	Frequency Array / HashMap
Character frequency	HashMap
Longest substring without repeat	Sliding Window
Longest substring satisfying a condition	Sliding Window
Pattern searching	KMP / Rabin-Karp
Common prefix	Trie / Comparison
Subsequences	Recursion / DP
Generate all substrings	Nested loops
String transformation	DP
Minimum edits	Edit Distance DP

3. Two Pointers

Use when

- Array or string is sorted.
- You need two elements satisfying a condition.
- You need to process from both ends.

Pattern	Use
Pair sum	Left + Right
Palindrome	Left + Right
Container with water	Left + Right

3Sum	Sort + Two Pointers
Remove duplicates	Slow + Fast
Partition array	Two Pointers

4. Sliding Window

Use when

The problem involves a **subarray** or **substring** and asks for a maximum, minimum, or count.

Fixed-size window

Use when the window size is given.

Example: maximum sum of subarray of size `k`.

Variable-size window

Use when you need the longest or shortest window satisfying a condition.

Examples:

- Longest substring without repeating characters
- Minimum size subarray sum
- Longest subarray with at most `k` distinct values

5. Binary Search

Use when

- Array is sorted.
- Search space is monotonic.
- You need minimum or maximum possible answer.

Pattern	Example
---------	---------

Find element	Classic Binary Search
First occurrence	Lower Bound
Last occurrence	Upper Bound
Rotated array	Modified Binary Search
Minimum in rotated array	Binary Search
Peak element	Binary Search
Square root	Binary Search on Answer
Koko Eating Bananas	Binary Search on Answer
Allocate Books	Binary Search on Answer

Recognition rule

Ask:

Can I check whether an answer x is possible?

If yes, think **Binary Search on Answer**.

6. Recursion

Use when

A problem can be broken into a smaller version of itself.

Common patterns:

Pattern	Example
Include / Exclude	Subsequences
Choose one	Permutations

Divide problem	Merge Sort
Tree traversal	DFS
Try all possibilities	Backtracking

7. Backtracking

Use when

Try every possible choice and undo the choice.

Pattern:

```
for (choice : choices) {  
    choose();  
    solve();  
    undo();  
}
```

Use for:

- Permutations
- Combinations
- Subsets
- N-Queens
- Sudoku
- Word Search
- Generate Parentheses

8. Linked List

Problem	Technique
Reverse list	Three pointers

Find middle	Slow/Fast pointers
Detect cycle	Floyd's Cycle Detection
Find cycle start	Floyd's Algorithm
Merge sorted lists	Two pointers
Remove nth node	Two pointers
Intersection	Two pointers
Palindrome list	Middle + Reverse

Remember

- Slow moves 1 step.
- Fast moves 2 steps.

Use for:

- Finding middle
- Detecting cycles

9. Stack

Use when

You need reverse processing or need to remember unresolved elements.

Problem	Pattern
Valid parentheses	Stack
Next greater element	Monotonic Stack
Next smaller element	Monotonic Stack
Largest rectangle	Monotonic Stack
Stock span	Monotonic Stack

Remove adjacent duplicates	Stack
Expression evaluation	Stack

10. Queue / Deque

Pattern	Use
BFS	Queue
Level order traversal	Queue
Sliding window maximum	Deque
Process tasks in order	Queue

11. Hashing

Use when

You repeatedly need to check whether something already exists.

Need	Use
Frequency	HashMap
Existence	HashSet
Index lookup	HashMap
Count subarrays	Prefix Sum + HashMap
Two Sum	HashMap
Duplicate detection	HashSet

12. Prefix Sum

Use when

You need repeated answers about sums of subarrays.

```
prefix[i] = prefix[i - 1] + arr[i];
```

Subarray sum:

```
prefix[r] - prefix[l - 1]
```

Common patterns:

- Range Sum
- Subarray Sum K
- Equal number of 0s and 1s
- Count subarrays

13. Bit Manipulation

Core tricks


```

// Check odd
n & 1

// Check kth bit
n & (1 << k)

// Set kth bit
n | (1 << k)

// Clear kth bit
n & ~(1 << k)

// Toggle kth bit
n ^ (1 << k)

// Remove rightmost set bit
n & (n - 1)

// Get rightmost set bit
n & (-n)

```

Need	Trick
Check odd	<code>n & 1</code>
Check kth bit	<code>n & (1 << k)</code>
Set bit	<code>n (1 << k)</code>
Clear bit	<code>n & ~(1 << k)</code>
Toggle bit	<code>n ^ (1 << k)</code>
Remove rightmost 1	<code>n & (n - 1)</code>
Get rightmost 1	<code>n & (-n)</code>
Power of 2	<code>n > 0 && (n & (n - 1)) == 0</code>
Count set bits	Repeatedly <code>n &= n - 1</code>

Unique element	XOR
All subsets	Bitmask

14. Trees

Main rule

- Need depth exploration → DFS
- Need level exploration → BFS

DFS Traversals

Traversal	Order
Preorder	Root Left Right
Inorder	Left Root Right
Postorder	Left Right Root

15. Binary Search Trees

Property

Left < Root < Right

Problem	Technique
Search	BST property
Insert	BST property
Sorted output	Inorder
Kth smallest	Inorder

LCA	Compare values
Validate BST	Range checking

16. Heap / Priority Queue

Use when

You repeatedly need the smallest or largest element.

Need	Heap
Largest repeatedly	Max Heap
Smallest repeatedly	Min Heap
Top K largest	Min Heap
Top K smallest	Max Heap
K closest	Heap
Merge K lists	Min Heap
Median stream	Two Heaps

17. Greedy

Use when

A locally optimal choice leads to a globally optimal solution.

Common clues:

- Minimum operations
- Maximum profit
- Interval scheduling
- Sorting may help

Problem	Greedy idea
Activity selection	Earliest finish
Fractional knapsack	Highest value/weight
Jump Game	Furthest reachable
Gas Station	Greedy
Merge intervals	Sorting

18. Intervals

First step is often

```
sort(intervals.begin(), intervals.end());
```

Then compare the current interval with the previous interval.

Use for:

- Merge intervals
- Insert interval
- Meeting rooms
- Minimum arrows
- Interval scheduling

19. Graphs

Need	Use
Level / shortest unweighted path	BFS
Explore completely	DFS
Connected components	DFS/BFS

Cycle detection	DFS/BFS
Shortest weighted path	Dijkstra
Negative edges	Bellman-Ford
All-pairs shortest path	Floyd-Warshall
Dependency ordering	Topological Sort
Minimum spanning tree	Kruskal / Prim

20. BFS

Use when

- Shortest path in an unweighted graph
- Minimum number of steps
- Level-by-level traversal
- Nearest distance

Template:

```
queue<int> q;

q.push(start);
visited[start] = true;

while (!q.empty()) {
    int node = q.front();
    q.pop();

    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            q.push(neighbor);
        }
    }
}
```

21. DFS

Use when

- Explore all paths
- Connected components
- Cycle detection
- Backtracking

```
void dfs(int node) {  
    visited[node] = true;  
  
    for (int neighbor : graph[node]) {  
        if (!visited[neighbor]) {  
            dfs(neighbor);  
        }  
    }  
}
```

22. Dynamic Programming

Main recognition rule

Ask:

Does the same subproblem get solved repeatedly?

If yes, think **Dynamic Programming**.

Pattern	Example
1D DP	Climbing Stairs
2D DP	Grid paths
Take / Not Take	Knapsack
Subsequences	LIS/LCS
Strings	Edit Distance
Partition	Matrix Chain Multiplication
Bitmask DP	Traveling Salesman
Tree DP	House Robber III

23. Knapsack Pattern

Use when

The problem involves:

- Choosing items
- Maximum/minimum value
- Limited capacity or target

Core decision:

Take
OR
Don't take

24. LIS Pattern

Use when

The problem asks about:

- Longest increasing sequence
- Increasing subsequence
- Maintaining order

Think:

- DP
- Binary Search optimization

25. LCS Pattern

Use when

The problem involves:

- Two strings or sequences
- Longest common elements
- Maintaining relative order

Think:

2D Dynamic Programming

26. Matrix / Grid

Question	Technique
Reach all cells	DFS/BFS
Shortest path	BFS
Count islands	DFS/BFS
Multi-source spread	Multi-source BFS
Count paths	DP
Rotate matrix	Transpose + Reverse
Spiral traversal	Boundary traversal

27. Trie

Use when

The problem involves many words and prefixes.

Use for:

- Prefix search
- Autocomplete
- Dictionary

- Word search
- Maximum XOR

28. Union Find / DSU

Use when

You repeatedly need to know whether two elements belong to the same group.

Use for:

- Connected components
- Cycle detection
- Kruskal's algorithm
- Dynamic connectivity
- Number of provinces

Core operations:

```
find(x)
union(x, y)
```

MASTER PATTERN RECOGNITION TABLE

If you see	Immediately think
Pair in sorted array	Two Pointers
Longest substring	Sliding Window
Count subarrays	Prefix Sum + HashMap
Next greater/smaller	Monotonic Stack
Top K	Heap
Minimum steps	BFS

Connected components	DFS/BFS
Sorted + answer is monotonic	Binary Search
Try all combinations	Backtracking
Repeated subproblems	DP
Many prefix searches	Trie
Groups/components merge	DSU
Binary/set bits	Bit Manipulation
Intervals	Sort + Compare

FINAL PROBLEM-SOLVING CHECKLIST

When reading a DSA problem, ask:

1. What data structure is involved?
2. Is it asking for:
 - Pair?
 - Subarray?
 - Substring?
 - Shortest path?
 - All possibilities?
3. Can sorting help?
4. Do I need repeated lookup?
 - HashMap / HashSet
5. Is the answer monotonic?
 - Binary Search
6. Are subproblems repeating?
 - Dynamic Programming
7. Do I need to explore choices?
 - Recursion / Backtracking
8. Do I need nearest/minimum steps?
 - BFS
9. Do I need next greater/smaller?
 - Monotonic Stack
10. Do I repeatedly need top/smallest/largest?
 - Heap

THE MOST IMPORTANT PATTERNS TO MASTER FIRST

1. Arrays + Hashing
2. Two Pointers
3. Sliding Window
4. Binary Search
5. Linked Lists
6. Stack + Monotonic Stack
7. Recursion + Backtracking
8. Trees
9. Heap
10. Graph BFS/DFS
11. Dynamic Programming
12. Greedy
13. Bit Manipulation
14. Trie
15. Union Find / DSU